

Plano de Apresentação: Docker e Cloud Computing para Desenvolvimento de Software

Docente: João Cardoso

23 de julho de 2026

Informações Gerais da Apresentação

- **Título:** Docker e Cloud Computing para Desenvolvimento de Software
- **Duração:** 15-20 minutos
- **Número de Slides:** 10
- **Público-alvo:** Estudantes de ADS, desenvolvedores iniciantes
- **Objetivo:** Apresentar os conceitos de containers, Docker e sua integração com a nuvem AWS
- **Sumário dos Assuntos e Plano do Curso:** <https://github.com/Zuluke/softwarearch-cloudcomputing>

Estrutura da Apresentação

Slide	Seção	Título
1	Problema	O Problema: Cozinhas Diferentes, Mesmo Prato
2	VM vs Container	VM vs Container: Diferenças de Arquitetura
3	Container	Containers: O Coração da Reprodutibilidade (Diagrama)
4	Container	Containers: O Coração da Reprodutibilidade (Benefícios)
5	Docker na Prática	Docker na Prática: Linhas Importantes
6	Exemplo	Exemplo: Containerizando uma API Flask (app.py + Dockerfile)
7	Exemplo	Exemplo: Containerizando uma API Flask (Comandos)
8	Desafio	Desafio: Containerize seu Próprio Código (Instruções)
9	Desafio	Desafio: Containerize seu Próprio Código (Critérios)
10	Conclusão	Escalabilidade na Nuvem (AWS)

Mapeamento dos Slides para o Cronograma

Slide	Tópico no Cronograma	Módulo/Item
1	Problema da consistência em equipes	Módulo I - Item 2 (Requisitos Não Funcionais)
2	Virtualização x Containers	Módulo IV - Item 11 (Virtualização)
3	Arquitetura de Containers	Módulo V - Item 14 (Containers)
4	Benefícios dos Containers	Módulo V - Item 14 (Containers)
5	Dockerfile	Módulo V - Item 14 (Dockerfile)

6	Exemplo Flask + Dockerfile	Módulo V - Item 14 (Containers)
7	Comandos Docker	Módulo V - Item 14 (Containers)
8	Desafio de Containerização	Módulo VIII - Item 24 (Implementação)
9	Critérios de Avaliação	Módulo VIII - Item 26 (Apresentação Final)
10	Escalabilidade na Nuvem (AWS)	Módulo VI - Item 19 (ECS, ECR, S3)

Mapeamento dos Slides para o Plano de Curso

Slide	Aula no Plano	Fase
1	Aula 1 - Containers e Virtualização	Parte 1: Exposição Teórica
2	Aula 1 - Containers e Virtualização	Parte 1: Exposição Teórica
3	Aula 1 - Containers e Virtualização	Parte 1: Exposição Teórica
4	Aula 1 - Containers e Virtualização	Parte 1: Exposição Teórica
5	Aula 2 - Dockerfile e Criação de Imagens	Parte 1: Exposição Teórica
6	Aula 2 - Dockerfile e Criação de Imagens	Parte 2: Prática Guiada
7	Aula 2 - Dockerfile e Criação de Imagens	Parte 2: Prática Guiada
8	Aula 2 - Dockerfile e Criação de Imagens	Prática Individual
9	Matriz de Avaliação	-
10	Aula 5/6 - AWS Introdução e ECR/ECS	Parte 1: Exposição Teórica

Roteiro Detalhado da Apresentação

Slide 1 — O Problema (1 min)

Contexto: Introduzir o problema da inconsistência entre ambientes de desenvolvimento.

- **Fala:** “Quantas vezes você já ouviu ‘funciona na minha máquina’? Esse é um dos maiores problemas no desenvolvimento de software.”
- **Analogia:** Explicar a analogia da cozinha (receita = código, ingredientes = bibliotecas, fogão = SO)
- **Exemplos:** Mostrar a diferença entre ambientes (Windows, Linux, macOS)
- **Solução:** Introduzir o conceito de container como “kit de cozinha portátil”

Base no Cronograma: Módulo I - Item 2 (Requisitos Não Funcionais)

Base no Plano: Aula 1 - Parte 1 (Exposição Teórica)

Slide 2 — VM vs Container (2 min)

Contexto: Comparar arquiteturas de Virtualização e Containers.

- **VM (Pesada):** Explicar SO Convidado, Hypervisor, hardware virtualizado
- **Container (Leve):** Explicar Docker Engine, compartilhamento de Kernel
- **Diagrama:** Mostrar visualmente as diferenças
- **Conclusão:** Containers são mais leves e rápidos

Base no Cronograma: Módulo IV - Item 11 (Virtualização)

Base no Plano: Aula 1 - Parte 1 (Exposição Teórica)

Slide 3 — Containers: Arquitetura (1.5 min)

Contexto: Detalhar a arquitetura de containers.

- **Diagrama:** Mostrar apps, bibliotecas, Docker Engine, Kernel
- **Kernel Compartilhado:** Todos os containers usam o mesmo kernel
- **Docker Engine:** Gerencia os containers
- **Hardware:** Camada física

Base no Cronograma: Módulo V - Item 14 (Containers)

Base no Plano: Aula 1 - Parte 1 (Exposição Teórica)

Slide 4 — Containers: Benefícios (1.5 min)

Contexto: Apresentar as vantagens dos containers.

- **Isolamento:** Cada container tem seu próprio sistema de arquivos, rede e processos
- **Leveza:** Sem SO convidado = economia de GBs
- **Velocidade:** Inicialização em segundos (vs. minutos das VMs)
- **Portabilidade:** Funciona em qualquer lugar com Docker instalado

Base no Cronograma: Módulo V - Item 14 (Containers)

Base no Plano: Aula 1 - Parte 1 (Exposição Teórica)

Slide 5 — Dockerfile (3 min)

Contexto: Explicar a estrutura do Dockerfile.

- **FROM:** Define a imagem base (ex: python:3.10-slim)
- **WORKDIR:** Define o diretório de trabalho
- **RUN:** Executa comandos durante o build (ex: apt-get, pip install)
- **COPY:** Copia arquivos para o container
- **ENTRYPOINT/CMD:** Define o comando de execução
- **Dica:** Multi-stage builds para otimização

Base no Cronograma: Módulo V - Item 14 (Dockerfile)

Base no Plano: Aula 2 - Parte 1 (Exposição Teórica)

Slides 6 e 7 — Exemplo API Flask (3 min)

Contexto: Mostrar um exemplo prático completo.

- **app.py:** Código Flask com rota '/'
- **Dockerfile:** Estrutura completa com FROM, WORKDIR, COPY, RUN, CMD
- **requirements.txt:** Dependências (flask)
- **Comandos:** docker build, docker run, curl
- **Resultado:** Hello, Docker!

Base no Cronograma: Módulo V - Item 14 (Containers)

Base no Plano: Aula 2 - Parte 2 (Prática Guiada)

Slides 8 e 9 — Desafio (2 min)

Contexto: Apresentar o desafio para os alunos.

- **Instruções:** Escolher um programa Python e containerizar
- **Entregáveis:** Dockerfile, código fonte, requirements.txt, prints
- **Critérios:** Dockerfile correto (40%), código funcional (40%), documentação (20%), criatividade (10%)
- **Prazo:** 1 semana

Base no Cronograma: Módulo VIII - Itens 24 e 26 (Implementação e Apresentação)

Base no Plano: Aula 2 - Prática Individual e Matriz de Avaliação

Slide 10 — Escalabilidade na Nuvem (2 min)

Contexto: Integrar containers com a nuvem AWS.

- **ECR:** Armazena imagens Docker com segurança
- **AWS Batch:** Orquestra e executa milhares de containers em paralelo

- **S3:** Armazena resultados finais
 - **Pipeline:** Código → Docker Build → Registry → AWS Batch
 - **Conclusão:** Containers garantem reprodutibilidade; Nuvem garante escala
- Base no Cronograma:** Módulo VI - Itens 17 e 19 (CI/CD e Serviços AWS)
- Base no Plano:** Aulas 5 e 6 - Parte 1 (Exposição Teórica)

Material de Apoio para o Instrutor

Distribuição do Tempo

Seção	Slides	Tempo	Atividade do Instrutor
Introdução	1	1 min	Contextualizar o problema
Conceitos	2-4	5 min	Explicar arquitetura e benefícios
Prática	5-7	6 min	Mostrar Dockerfile e exemplo
Desafio	8-9	2 min	Explicar atividade
Conclusão	10	2 min	Integrar com nuvem e finalizar
Total	10	16 min	-

Dicas de Apresentação

- **Slide 1:** Use a analogia da cozinha para engajar a turma
- **Slides 2-4:** Enfatize a diferença de arquitetura com diagramas
- **Slides 5-7:** Mostre o código em tempo real se possível
- **Slides 8-9:** Reforce que o desafio é individual e tem prazo de 1 semana
- **Slide 10:** Conecte com o mercado (AWS é a nuvem mais usada)

Perguntas Frequentes

- **Qual a diferença entre VM e Container?** VM tem SO completo; Container compartilha kernel
- **O que é Dockerfile?** Receita para construir uma imagem
- **O que é ECR?** Repositório de imagens Docker na AWS
- **O que é AWS Batch?** Orquestrador de jobs em lote

Referências para Aprofundamento

- Mark Richards and Neal Ford, “Fundamentals of Software Architecture”. O’Reilly, 1st Ed., 2020.
- Documentação Docker: <https://docs.docker.com/>
- Documentação AWS ECR: <https://docs.aws.amazon.com/ecr/>
- Documentação AWS Batch: <https://docs.aws.amazon.com/batch/>

Checklist de Pré-Apresentação

- ☐ Testar execução do código em tempo real (se possível)
- ☐ Preparar ambiente Docker para demonstração (se for demonstrar)
- ☐ Verificar conexão com AWS (se for demonstrar)

Observações Finais

- **Contexto da Disciplina:** Esta apresentação faz parte do Módulo V (Containers) do curso de Arquitetura de Software e Computação em Nuvem
- **Pré-requisitos:** Os alunos já tiveram introdução a fundamentos de arquitetura (Módulo I) e modelagem UML (Módulo II)
- **Próximos Passos:** Após esta apresentação, os alunos iniciarão a prática com Docker e, em seguida, avançarão para Kubernetes (Módulo V.16)
- **Integração:** Os conceitos apresentados serão utilizados no Projeto Integrador (Módulo VIII)